L'Architecture Client-Serveur et le Protocole HTTP

Thème principal : Réseaux et Protocoles Web

Synthèse exécutive : Ce cours pose les fondements indispensables du fonctionnement du Web en détaillant l'architecture client-serveur et le protocole HTTP (HyperText Transfer Protocol). L'étudiant y découvre comment un navigateur formule des requêtes standardisées pour obtenir des ressources auprès d'un serveur web, comment ce dernier y répond de manière asynchrone et sans état (stateless), et comment décoder les codes de statut HTTP (2xx, 3xx, 4xx, 5xx) pour diagnostiquer et déboguer efficacement des applications web.

L'Architecture Client-Serveur et le Principe Stateless

L'architecture client-serveur est le modèle de communication fondamental du Web. Contrairement au pair-à-pair (P2P), ce modèle établit une distinction stricte des rôles : le client initie toujours la communication par une requête, tandis que le serveur attend passivement. Une caractéristique essentielle de HTTP est son absence d'état (stateless) : le serveur oublie instantanément le client une fois la réponse envoyée. Pour maintenir des sessions (comme un panier d'achat), des mécanismes comme les cookies et les sessions sont indispensables.

- Client (Navigateur) : Logiciel actif qui prend l'initiative d'envoyer une requête pour obtenir une ressource.
- Serveur Web : Machine ou processus logiciel passif qui écoute sur un port spécifique (ex: 80 ou 443) et traite les requêtes entrantes.
- Protocole Stateless (Sans état) : Caractéristique fondamentale de HTTP où chaque échange requête-réponse est indépendant, sans mémoire des interactions passées.

Cas concret enseignant : *Le professeur compare HTTP à un guichetier amnésique : chaque fois que vous vous présentez devant lui, même si c'était il y a deux secondes, il vous traite comme un parfait inconnu. Pour qu'il se souvienne de vous, vous devez lui présenter un badge (le cookie de session) à chaque phrase.*

Structure des Requêtes, Réponses et Méthodes HTTP

Une requête ou réponse HTTP est un message textuel structuré comportant une première ligne descriptive, des en-têtes (headers), une ligne vide obligatoire, et un corps (body) optionnel. Les méthodes GET et POST constituent les deux verbes les plus utilisés : GET récupère une ressource sans modifier l'état du serveur (méthode sûre et idempotente), tandis que POST transmet des données de manière invisible dans le corps pour altérer l'état du système.

- Ligne de requête / statut : Contient la méthode, l'URI et la version pour la requête, ou le code de statut et sa description pour la réponse.
- En-têtes (Headers) : Métadonnées sous forme de clés-valeurs (Host, User-Agent, Content-Type) décrivant le contexte.
- Méthodes GET vs POST : GET est pour la consultation (paramètres dans l'URL), POST est pour la modification (paramètres dans le corps de la requête).

Cas concret enseignant : *Le professeur insiste sur le fait que le protocole HTTP est purement textuel : on peut littéralement lire les requêtes et réponses en clair si elles ne sont pas chiffrées par HTTPS. C'est pourquoi l'utilisation de GET pour des données sensibles comme un mot de passe est une faille majeure visible dans l'historique du navigateur.*

La Classification des Codes de Statut HTTP

Chaque réponse HTTP commence par un code numérique à trois chiffres permettant de classifier immédiatement le résultat. Il est crucial pour un développeur de distinguer les erreurs 4xx (problème imputable au client, comme une URL erronée) des erreurs 5xx (problème interne au serveur, comme un script défectueux ou une base de données inaccessible).

• Codes 2xx (Succès) : Indiquent le succès de la requête (ex: 200 OK).
• Codes 3xx (Redirection) : Indiquent qu'une action supplémentaire est requise.
• Codes 4xx (Erreur Client) : Indiquent une anomalie dans la requête du client (ex: 404 Not Found, 403 Forbidden).
• Codes 5xx (Erreur Serveur) : Indiquent une défaillance interne du serveur (ex: 500 Internal Server Error).

Cas concret enseignant : *Le professeur raconte qu'à ses débuts, il avait configuré par erreur une base de données qui plantait à chaque soumission de formulaire, renvoyant systématiquement une erreur 500. Les utilisateurs pensaient que le site était piraté alors que c'était simplement un point-virgule manquant dans son script PHP.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Monsieur, quelle est la différence concrète entre GET et POST pour envoyer un formulaire de connexion ?
Réponse du professeur : GET transmet les paramètres directement dans l'URL (visibles dans la barre d'adresse et l'historique, taille limitée), tandis que POST transmet les données de manière invisible dans le corps (body) de la requête, ce qui est indispensable pour la sécurité relative des formulaires de connexion et le transfert de volumes importants.

■ Exercice & Démonstration d'Application

Énoncé : Analyser la requête HTTP brute suivante et identifier la méthode utilisée, la ressource demandée, l'hôte cible, et expliquer ce que le serveur va renvoyer si la ressource n'existe pas : GET /images/logo.png HTTP/1.1 Host: iutbayonne.univ-pau.fr User-Agent: Mozilla/5.0

Résolution pas à pas :
1. Analyser la première ligne : 'GET /images/logo.png HTTP/1.1'. Le mot 'GET' est la méthode HTTP de récupération de ressource. L'URI cible est '/images/logo.png'. 2. Identifier l'hôte cible en lisant l'en-tête 'Host: iutbayonne.univ-pau.fr', qui indique le domaine sollicité. 3. Analyser l'en-tête 'User-Agent: Mozilla/5.0' qui identifie le navigateur client. 4. Déduire le comportement en cas d'absence : si le fichier logo.png n'est pas présent sur le serveur, celui-ci renvoie un code de statut de la classe 4xx, précisément '404 Not Found'.

Résultat : Méthode : GET Ressource : /images/logo.png Hôte cible : iutbayonne.univ-pau.fr Code de retour en cas d'absence : 404 Not Found (Erreur Client)

■ Auto-Évaluation Flash d'Examen

Q : Qu'est-ce qu'un protocole 'stateless' comme HTTP ?

R : Un protocole sans état où chaque échange requête-réponse est totalement indépendant et où le serveur ne conserve aucune mémoire des interactions passées.

Q : Quelle est la différence majeure entre les codes d'erreur de la série 4xx et ceux de la série 5xx ?

R : Les erreurs 4xx proviennent d'une anomalie de la requête du client (ex: URL fausse), tandis que les erreurs 5xx proviennent d'une défaillance interne du serveur (ex: bug de script ou plantage de base de données).

Q : Pourquoi n'utilise-t-on jamais la méthode GET pour transmettre un mot de passe ?

R : Parce que les paramètres de la méthode GET sont inscrits en clair dans l'URL, ce qui les rend visibles dans l'historique du navigateur, les logs du serveur et exposés au 'shoulder surfing'.